

AI Network Fault Prediction System - Complete Project Report

A Machine Learning Based Telecom Network Fault Prediction System

Using Random Forest, XGBoost, and LSTM Models

Author: Sai

Date: March 2026

Built with Python, Flask, Streamlit, scikit-learn, XGBoost, PyTorch

Table of Contents

- 1. Project Overview**
- 2. Technology Stack & Dependencies**
- 3. Project Structure**
- 4. Step-by-Step Implementation**
 - 4.1 Step 1: Project Setup
 - 4.2 Step 2: Synthetic Data Generation
 - 4.3 Step 3: Data Preprocessing & Feature Engineering
 - 4.4 Step 4: Model Training & Evaluation
 - 4.5 Step 5: Prediction & Anomaly Detection Module
 - 4.6 Step 6: Flask REST API
 - 4.7 Step 7: Streamlit Dashboard
 - 4.8 Step 8: Unit Testing
 - 4.9 Step 9: Running the System
- 5. Errors Encountered & Solutions**
- 6. Data Collection & Processing Pipeline**
- 7. Model Results & Comparison**
- 8. Conclusion**

1. Project Overview

The AI Network Fault Prediction System is a complete machine learning pipeline designed to predict network faults in telecom infrastructure before they occur. The system monitors network metrics such as latency, packet loss, signal strength, CPU utilisation, memory utilisation, error rates, and traffic volume from multiple network nodes.

The system uses three machine learning models - Random Forest, XGBoost, and LSTM (Long Short-Term Memory) neural network - to predict faults based on degraded network conditions. It also includes an Isolation Forest based anomaly detection system for identifying unusual network patterns.

Key Features

- * Synthetic telecom dataset generation with realistic fault patterns
- * Data preprocessing with rolling window features and cyclical time encoding
- * Three ML models: Random Forest, XGBoost, and LSTM
- * Anomaly detection using Isolation Forest
- * REST API (Flask) for real-time fault prediction
- * Interactive dashboard (Streamlit) with 5 pages for monitoring
- * SQLite database for storing predictions and alerts
- * Automated alert generation with severity levels (low/medium/high/critical)
- * Unit tests for data pipeline and API endpoints

2. Technology Stack & Dependencies

Library	Version	Purpose
Python	3.10+	Core language
pandas	>= 2.0.0	Data manipulation
numpy	>= 1.24.0	Numerical computation
scikit-learn	>= 1.3.0	ML models & preprocessing
xgboost	>= 2.0.0	Gradient boosting model
torch (PyTorch)	>= 2.0.0	LSTM neural network
flask	>= 3.0.0	REST API server
flask-cors	>= 4.0.0	Cross-origin support
streamlit	>= 1.28.0	Interactive dashboard
plotly	>= 5.18.0	Charts & visualisations
joblib	>= 1.3.0	Model serialization
requests	>= 2.31.0	HTTP client
pytest	>= 7.4.0	Unit testing

3. Project Structure

```
AI Telecom Fault Prediction/
|
|-- config.py           # Central configuration
|-- run.py             # CLI entry point
|-- requirements.txt    # Python dependencies
|
|-- data/
|   |-- __init__.py
|   |-- generate_dataset.py # Synthetic data generator
|   |-- preprocess.py     # Preprocessing pipeline
|   |-- raw/
|       |-- network_logs.csv # Generated raw data
|   |-- processed/
|       |-- train.csv        # Training set
|       |-- val.csv          # Validation set
|       |-- test.csv         # Test set
|
|-- models/
|   |-- __init__.py
|   |-- train.py            # Model training & evaluation
|   |-- predict.py         # Inference module
|   |-- anomaly.py         # Isolation Forest anomaly detection
|   |-- saved/
|       |-- best_model.joblib # Best performing model
|       |-- random_forest.joblib
|       |-- xgboost.joblib
|       |-- lstm.pt
|       |-- scaler.joblib    # StandardScaler
|       |-- metadata.json    # Model results
|
|-- api/
|   |-- __init__.py
|   |-- app.py             # Flask REST API
|   |-- database.py        # SQLite database helper
|   |-- alerts.py          # Alert generation logic
|   |-- network_faults.db  # SQLite database
|
|-- dashboard/
|   |-- __init__.py
|   |-- app.py             # Streamlit dashboard (multi-page)
|
|-- tests/
|   |-- __init__.py
|   |-- test_data.py       # Data pipeline tests
|   |-- test_api.py        # API endpoint tests
```

4. Step-by-Step Implementation

Step 1: Project Setup

The project was initialised by creating the directory structure and installing all required dependencies. A virtual environment was created for isolation.

Commands Executed:

```
# Create project directory
mkdir 'AI Telecom Fault Prediction'
cd 'AI Telecom Fault Prediction'

# Create virtual environment
python -m venv .venv
source .venv/bin/activate

# Install dependencies
pip install -r requirements.txt
```

A central configuration file (config.py) was created to store all paths, hyperparameters, feature column names, alert thresholds, and server settings in one place. This avoids hardcoded values scattered across the codebase.

Key Configuration Parameters:

- * NUM_SAMPLES = 50,000 synthetic records
- * NUM_NODES = 20 simulated network nodes
- * FAULT_RATIO = 15% fault samples
- * FAULT_PROB_THRESHOLD = 0.5 for classifying faults
- * API_PORT = 5050, DASHBOARD_PORT = 8501

Step 2: Synthetic Data Generation

Since real telecom network data is proprietary and difficult to obtain, a synthetic dataset generator was built to create realistic network monitoring data.

How the Data is Generated (generate_dataset.py):

1. Timestamps: 5-minute interval time series starting from 2024-01-01 for each of the 20 nodes (2,500 samples per node = 50,000 total).
2. Normal Baseline Metrics are generated using statistical distributions:
 - traffic_volume: Normal(500, 120), clipped at 50
 - latency_ms: Normal(20, 5), clipped at 1
 - packet_loss_pct: Exponential(0.5), clipped 0-100
 - signal_strength_dbm: Normal(-65, 8), clipped -120 to -30
 - error_rate: Exponential(0.02), clipped 0-1
 - cpu_utilisation: Normal(45, 15), clipped 0-100
 - memory_utilisation: Normal(50, 12), clipped 0-100
3. Fault Injection: Correlated anomaly windows of 6 consecutive samples are injected. Within each window, metrics are degraded (latency increases by ~80ms, packet loss by ~8%, error rate by ~0.15, CPU by ~25%, traffic multiplied by 1.5-2.5x, signal drops by ~15dBm). The fault label is set to 1 for the last 2 samples of each window.
4. The final DataFrame is saved as data/raw/network_logs.csv.

Command:

```
python run.py generate
```

Output:

```
Saved 50,000 rows to data/raw/network_logs.csv
Nodes: 20
Fault ratio: ~15%
Time range: 2024-01-01 00:00:00 -> 2024-01-09 15:55:00
```

Step 3: Data Preprocessing & Feature Engineering

The preprocessing pipeline (preprocess.py) performs the following steps:

3a. Rolling Window Features

For each of the selected columns (latency_ms, packet_loss_pct, error_rate), rolling mean and rolling standard deviation are computed per node with a window size of 5. This captures trends and volatility in the time series, which are critical indicators of impending faults.

3b. Cyclical Time Features

Hour-of-day and day-of-week are encoded as cyclical features using sine and cosine transformations. This preserves the circular nature of time (e.g., hour 23 is close to hour 0), adding 4 new features: hour_sin, hour_cos, dow_sin, dow_cos.

3c. Train / Validation / Test Split

The dataset is split into 70% training, 15% validation, and 15% test sets using stratified splitting to maintain the fault class ratio across all splits.

3d. Feature Scaling

StandardScaler (zero mean, unit variance) is fit on the training set and applied to validation and test sets. The fitted scaler is saved to models/saved/scaler.joblib for use during inference.

3e. NaN Handling

Any remaining NaN values (from rolling window edge effects) are filled with 0 using np.nan_to_num().

Final Feature Count: 17 features

7 original metrics + 6 rolling features (3 means + 3 stds) + 4 time features = 17 total.

Command:

```
python run.py preprocess
```


Step 4: Model Training & Evaluation

Three machine learning models are trained and evaluated on the test set. The best model is selected by F1-score.

Model 1: Random Forest

Configuration: 200 trees, max_depth=15, min_samples_split=5, class_weight='balanced', random_state=42. Uses parallel processing (n_jobs=-1).

Model 2: XGBoost

Configuration: 200 boosting rounds, max_depth=8, learning_rate=0.1, subsample=0.8, colsample_bytree=0.8, eval_metric='logloss'. Automatically computes scale_pos_weight from the training class distribution to handle imbalance.

Model 3: LSTM (PyTorch)

Architecture: 2-layer LSTM with hidden_size=64, dropout=0.3, followed by a fully connected layer (64 -> 32 -> 1). Trained with BCEWithLogitsLoss using pos_weight for class imbalance. The input data is reshaped into overlapping sequences of length 10. Trained for 30 epochs with Adam optimizer (lr=0.001), batch_size=256.

Evaluation Metrics:

Each model is evaluated using: Accuracy, Precision, Recall, F1-Score, Confusion Matrix.

Command:

```
python run.py train
```

All models and metadata (including which model was best) are saved to models/saved/. A metadata.json file stores the results for all three models.

Step 5: Prediction & Anomaly Detection Module

FaultPredictor (predict.py)

A singleton class that loads the best model, scaler, and feature names from metadata.json. It accepts a dictionary of network metrics, builds a feature vector, scales it, and returns the fault probability, is_fault flag, severity level, and model name. If LSTM was the best model, the system falls back to XGBoost for API serving (since LSTM requires sequence input).

AnomalyDetector (anomaly.py)

Uses Isolation Forest (contamination=0.1) to detect anomalous network patterns. Returns an anomaly score and a boolean is_anomaly flag.

Severity Levels:

Severity	Probability Range	Action
Normal	< 0.50	No action needed
Low	0.50 - 0.70	Monitor closely
Medium	0.70 - 0.85	Investigate soon
High	0.85 - 0.95	Immediate attention
Critical	0.95 - 1.00	Take action NOW

Step 6: Flask REST API

A Flask REST API (api/app.py) provides endpoints for real-time prediction and alert management. It uses SQLite (api/network_faults.db) for persistence.

API Endpoints:

Method	Endpoint	Description
GET	/health	Service health check - returns model name
POST	/predict	Predict fault from JSON network metrics
GET	/alerts	List recent alerts (filter by resolved)
POST	/alerts/<id>/resolve	Mark an alert as resolved
GET	/history	Historical predictions
GET	/metrics	Model performance metrics + live stats

Database Schema (SQLite):

Two tables:

- predictions: id, timestamp, node_id, fault_prob, is_fault, severity, model, features
- alerts: id, timestamp, node_id, severity, message, fault_prob, resolved

Command:

```
python run.py api
# API starts on http://0.0.0.0:5050
```

Step 7: Streamlit Dashboard

A multi-page Streamlit dashboard (dashboard/app.py) provides a visual interface for monitoring the network. It communicates with the Flask API for predictions and alerts.

Dashboard Pages:

- * Network Status - Real-time KPI gauges, time-series charts, CPU/Memory gauges per node
- * Fault Predictions - Submit metrics form, view recent predictions table
- * Alerts - Active alerts with resolve button, resolved alerts history
- * Model Performance - Comparison table & bar chart, confusion matrix, live stats
- * Historical Analysis - Fault timeline, per-node fault distribution, correlation heatmap

Command:

```
python run.py dashboard
# Dashboard opens at http://localhost:8501
```

Step 8: Unit Testing

Two test modules validate the core functionality:

test_data.py (Data Pipeline Tests):

- * TestDataGeneration: verifies shape, columns, binary labels, no NaNs, node count
- * TestPreprocessing: verifies rolling features, time features, preprocessing output

test_api.py (API Endpoint Tests):

- * Tests /health, /predict (valid + empty), /history, /alerts endpoints
- * Uses a temporary SQLite database via monkeypatch

Command:

```
pytest tests/ -v
```

Step 9: Running the Complete System

Quick Start (Everything at Once):

```
# Step 1: Install dependencies
pip install -r requirements.txt

# Step 2: Generate data, preprocess, and train (one command)
python run.py all

# Step 3: Start the API (Terminal 1)
python run.py api

# Step 4: Start the Dashboard (Terminal 2)
python run.py dashboard
```

Individual Commands:

Command	What It Does
python run.py generate	Generate 50K synthetic telecom records
python run.py preprocess	Preprocess data & create train/val/test splits
python run.py train	Train RF, XGBoost, LSTM & save best model
python run.py all	Run generate -> preprocess -> train in sequence
python run.py api	Start Flask API on port 5050
python run.py dashboard	Start Streamlit dashboard on port 8501

5. Errors Encountered & Solutions

Error 1: Port Already in Use

Error Message:

```
Port 8501 is already in use
```

Cause:

The Streamlit dashboard was already running on port 8501 from a previous session.

Solution:

```
# Kill the process using the port
lsof -ti:8501 | xargs kill -9

# OR specify a different port
streamlit run dashboard/app.py --server.port 8502
```

Error 2: Python 3.12 Compatibility - collections.abc

Error Message:

```
AttributeError: module 'collections' has no attribute 'Hashable'
```

Cause:

Python 3.10+ moved Hashable, Mapping, MutableMapping, MutableSet, and Callable from the collections module to collections.abc. Libraries like dask/xarray that Streamlit depends on had not yet been updated.

Solution (applied in dashboard/app.py):

```
import collections
import collections.abc
for attr in ("Hashable", "Mapping", "MutableMapping",
            "MutableSet", "Callable"):
    if not hasattr(collections, attr):
        setattr(collections, attr,
                getattr(collections.abc, attr))
```

Error 3: XGBoost use_label_encoder Deprecation

Error Message:

```
WARNING: use_label_encoder is deprecated in xgboost >= 2.0
```

Cause:

XGBoost 2.0+ removed the use_label_encoder parameter.

Solution:

The parameter was kept in config.py for backward compatibility but the warning is harmless. For newer versions of XGBoost, this parameter can be removed from XGBOOST_PARAMS.

Error 4: LSTM Fallback for API Serving

Issue:

LSTM models require sequence input (batch, seq_len, features) which makes single-sample real-time prediction via API impractical without maintaining a buffer of recent samples.

Solution:

When LSTM is the best model, the prediction module (predict.py) automatically falls back to XGBoost for API serving, since it accepts single feature vectors. The LSTM model is still saved and can be used in batch prediction scenarios.

Error 5: API Unreachable from Dashboard

Issue:

The Streamlit dashboard shows 'API unreachable' messages when the Flask API is not running.

Solution:

The API must be started BEFORE the dashboard. Run 'python run.py api' in one terminal, then 'python run.py dashboard' in a second terminal. The dashboard gracefully handles API failures by showing informational messages instead of crashing.

Error 6: Empty Request Body (400 Error)

Issue:

Sending an empty JSON body {} to /predict returns a 400 error.

Solution:

This is intentional validation. The API requires network metrics in the request body. The error handling was implemented to return a clear error message: 'No JSON body provided'.

6. Data Collection & Processing Pipeline

This section provides a detailed flow of how data moves through the system, from generation to prediction.

Stage 1: Data Generation

Synthetic data is generated using NumPy random distributions to simulate realistic telecom network monitoring. The key insight is the fault injection mechanism: correlated anomaly windows are created where MULTIPLE metrics degrade simultaneously (latency spikes, packet loss increases, signal drops, CPU rises). The fault label fires near the END of these degradation windows, simulating how real faults are preceded by warning signs. Output: data/raw/network_logs.csv (50,000 rows x 10 columns).

Stage 2: Feature Engineering

Rolling Features: For latency_ms, packet_loss_pct, and error_rate, 5-sample rolling mean and standard deviation are computed per node. This captures recent trends and variability.

Time Features: Hour and day-of-week are encoded as sin/cos pairs to preserve cyclical relationships.

Result: 17 features total (7 raw + 6 rolling + 4 cyclical time).

Stage 3: Splitting & Scaling

The data is split 70/15/15 (train/val/test) with stratification to maintain class balance. StandardScaler is fit on training data only and applied to all sets. This prevents data leakage. The scaler and feature names are persisted for inference.

Stage 4: Model Training

Three models are trained on the same data. Random Forest and XGBoost handle class imbalance via class_weight='balanced' and scale_pos_weight respectively. LSTM uses BCEWithLogitsLoss with pos_weight. All models are evaluated on the test set using accuracy, precision, recall, and F1-score.

Stage 5: Inference Pipeline

When a prediction request arrives at the API:

1. Raw metrics are extracted from the JSON body
2. A feature vector is built in the correct column order
3. Missing features (rolling/time) are filled with 0.0
4. The vector is scaled using the saved StandardScaler
5. The best model predicts fault probability
6. Severity is determined from probability thresholds
7. Isolation Forest checks for anomalies
8. Result is saved to SQLite and an alert is created if needed

7. Model Results & Comparison

All three models were trained on 35,000 samples and evaluated on 7,500 test samples. The following results are typical outputs (exact values depend on the random seed).

Model	Accuracy	Precision	Recall	F1-Score
Random Forest	0.96+	0.88+	0.80+	0.84+
XGBoost	0.97+	0.90+	0.85+	0.87+
LSTM	0.95+	0.85+	0.78+	0.81+

Note: The exact values above are approximate. XGBoost typically performs best on this dataset due to its ability to handle tabular data with gradient boosting. The LSTM model requires sequence input and its performance is affected by the sequence creation process.

Why XGBoost Performs Best

- * Gradient boosting is well-suited for tabular data with mixed feature types
- * `scale_pos_weight` effectively handles the class imbalance (85/15 split)
- * Feature interactions are captured naturally by tree-based splitting
- * Regularization (`subsample`, `colsample_bytree`) prevents overfitting

Model Selection & Saving

The model with the highest F1-score is automatically selected as the 'best model' and saved as `best_model.joblib`. All three models are also saved individually for flexibility. A `metadata.json` file records all results.

8. Conclusion

The AI Network Fault Prediction System demonstrates a complete end-to-end machine learning pipeline for telecom network monitoring. Key achievements include:

- * Successfully generated realistic synthetic telecom data with correlated fault patterns
- * Implemented comprehensive feature engineering with rolling windows and cyclical encoding
- * Trained and compared three ML models (Random Forest, XGBoost, LSTM)
- * Built a production-ready REST API with Flask for real-time predictions
- * Created an interactive 5-page Streamlit dashboard for network monitoring
- * Implemented automated alert generation with severity classification
- * Added anomaly detection using Isolation Forest
- * Wrote unit tests covering data pipeline and API endpoints
- * Handled Python 3.12 compatibility issues and model serving challenges

The system is designed for extensibility. Future improvements could include:

- Integration with real network monitoring tools (SNMP, Prometheus)
- Real-time streaming with Apache Kafka or Redis
- Containerisation with Docker for deployment
- More advanced models (Transformers, Graph Neural Networks)
- Email/SMS notification integration for alerts

--- End of Report ---